Linux Commands

1. sha256sum

Calculates a SHA-256 checksum for a file. This can help verify whether file contents have changed.

Command:

```
sha256sum file1.txt
```

Example output:

```
<checksum>  file1.txt
```

Explanation: The long hexadecimal value is the checksum. If the file content changes, the checksum will usually change too.

2. du -ah ~ --max-depth=1

Displays disk usage for items directly inside your home directory, including hidden files and directories.

Command:

```
du -ah ~ --max-depth=1
```

Example output:

```
4.0K  /home/Srini/file1.txt
60K   /home/Srini/.cache
196K  /home/Srini
```

Explanation: -h shows human-readable sizes, -a includes files as well as directories, and --max-depth=1 limits the displayed depth. Sizes vary.

3. ls -lt

Lists directory contents with the most recently modified entries first.

Command:

```
ls -lt
```

Example output:

```
-rw-r--r-- ... Oct 28 09:36 xx01
-rw-r--r-- ... Oct 28 09:36 xx00
```

Explanation: -l shows detailed information; -t sorts by modification time, newest first. Exact output depends on the directory.

4. xargs -n 1 echo < names.txt

Reads input from names.txt and passes one whitespace-separated item at a time to echo.

Command:

```
xargs -n 1 echo < names.txt
```

Example output:
I
am
currently
doing
my
work...

Explanation: -n 1 limits each echo invocation to one input item. By default, xargs splits input on whitespace, so phrases containing spaces are split into separate items.

5. expand -t 4

Converts tab characters into spaces, using tab stops every four columns.

Command:

expand -t 4 file1.txt

Example:
A\tB\tC → A B C

Explanation: The spaces inserted depend on the current column and the next tab stop. expand prints converted text and does not modify the original file.

6. unexpand -a

Converts suitable sequences of spaces into tab characters, wherever possible.

Command:

unexpand -a file1.txt
unexpand -a file1.txt | cat -T

The second command displays tabs as ^I.

Explanation: The output may look similar to the original, but it is not guaranteed to recreate the exact original tabs. The original file remains unchanged unless you redirect output.

7. od -c

Displays file contents in a character-oriented representation, useful for inspecting tabs and newline characters.

Command:

od -c file1.txt

Markers:
\t tab character
\n newline character

Explanation: The leftmost values are byte offsets. This output helps distinguish actual tab characters from ordinary spaces.

8. split -l 2

Splits a file into multiple smaller files, each containing up to two lines.

Command:

```
split -l 2 file1.txt chunk_
ls -l chunk_*
cat chunk_aa
```

Example files:
chunk_aa
chunk_ab
chunk_ac

Explanation: -l 2 means two lines per output file. The final file may contain fewer lines. The prefix chunk_ is followed by suffixes such as aa, ab, and ac.

9. csplit

Splits a file at a specified line number or pattern and creates output files named xx00, xx01, and so on by default.

Command:

```
csplit file1.txt 3
cat xx00
cat xx01
```

If file1.txt contains:
Hi
Hello
How
are
you?

xx00:
Hi
Hello

xx01:
How
are
you?

Explanation: The numeric pattern 3 splits immediately before line 3. csplit prints the byte size of each output piece. It does not edit file1.txt. Existing xx00/xx01 files may be overwritten.

10 . history 10

Displays the last 10 commands